复现SemGauss-SLAM

Created by	Haotian Zhou
Last edited by	Su Wangjie
Last edited time	@2026年1月11日 10:55
Status	In progress

遇到的问题：

1. SemGauss环境配置，老旧环境不适配新系列显卡
2. SemGuass运行效率与效果兼顾
3. SemGuass结果评价指标与后处理
4. 理解semantic真值来源为何，对精度、质量有何影响；是否使用定位真值的影响
5. SemGuass新场景及无GT语义场景下的运行探索
6. 可视化未知视角的渲染图像，参考3DGS原始仓库的SIBR viewer，或其它查看3DGS表示的工具
7. mIoU结果异常
8. SIBR可视化异常
9. 测试GOAT-Core数据

与作者库直接相关的问题

CUDA 非法内存访问报错

Traceback (most recent call last):
File "/home/csxq/tasks/SemGauss-SLAM/sem_gauss.py", line 1028, in <module>
dense_semantic_slam(experiment.config)
File "/home/csxq/tasks/SemGauss-SLAM/sem_gauss.py", line 778, in dense_semantic_slam
iter_data = {'cam': cam, 'im': iter_color.to('cuda',non_blocking=True), 'depth': iter_depth.to('cuda',non_blocking=True),
torch.AcceleratorError: CUDA error: an illegal memory access was encountered

▼ 我们发现在我们迁移版本后的库中, 在运行时有很大概率发生以上错误, 以下是我们逐步发现并解决这个问题的方案

- 使用 `CUDA_LAUNCH_BLOCKING=1` 环境变量使得报错同步发生
  - 在使用了这个环境变量后, 我们可以定位到发生错误的位置应该在反向传播附近, 但是 **依然看不到具体的 kernel 调用**
  - 因此我们推断问题应该发生在作者自己完成的 cuda kernel 函数中
- 我们尝试了重编译库 `diff-gaussian-rasterization` 并不能真的解决问题
- 接下来我们使用了 `compute-sanitizer` 这个 NVIDIA 的分析工具, 为了避免和 Pytorch 本身的缓存机制冲突, 我们额外引入了 `export PYTORCH_NO_CUDA_MEMORY_CACHING=1` 这个环境变量

▼ 此时在第一个 iteration 就看到了核函数准确的报错信息!

```
========= Invalid __global__ read of size 4 bytes

=========     at void renderCUDA<(unsigned int)6, (unsigned int)16>(const uint2 *, cons
t unsigned int *, int, int, const float *, const float2 *, const float4 *, const float
*, const float *, const unsigned int *, const float *, float3 *, float4 *, float *, flo
at *, const float *, const float *, float *)+0x6c0 in backward.cu:469

=========     by thread (0,10,0) in block (1,42,0)
```

```
=========     Address 0x7876731d05c0 is out of bounds

=========     and is 9,665 bytes after the nearest allocation at 0x787670000000 of size
52,224,000 bytes

=========     Saved host backtrace up to driver entry point at kernel launch time

=========         Host Frame:  [0x13e88] in libcudart.so.12

=========         Host Frame: cudaLaunchKernel [0x79f87] in libcudart.so.12

=========         Host Frame: BACKWARD::render(dim3, dim3, uint2 const*, unsigned int c
onst*, int, int, float const*, float2 const*, float4 const*, float const*, float const
*, unsigned int const*, float const*, float3*, float4*, float*, float*, float const*, f
loat const*, float*) [0x29bb07] in _C.cpython-310-x86_64-linux-gnu.so

=========         Host Frame: CudaRasterizer::Rasterizer::backward(int, int, int, int,
float const*, int, int, float const*, float const*, float const*, float const*, float,
float const*, float const*, float const*, float const*, float const*, float, float, int
const*, char*, char*, char*, float const*, float*, float*, float*, float*, float*, floa
t*, float*, float*, float*, float const*, float const*, float*) [0x2aaaaf] in _C.cpytho
n-310-x86_64-linux-gnu.so

=========         Host Frame: RasterizeGaussiansBackwardCUDA(at::Tensor const&, at::Ten
sor const&, at::Tensor const&, at::Tensor const&, at::Tensor const&, at::Tensor const&,
float, at::Tensor const&, at::Tensor const&, at::Tensor const&, float, float, at::Tenso
r const&, at::Tensor const&, int, at::Tensor const&, at::Tensor const&, int, at::Tensor
const&, at::Tensor const&, at::Tensor const&, at::Tensor const&) [0x2d4d16] in _C.cpyth
on-310-x86_64-linux-gnu.so

=========         Host Frame: pybind11::cpp_function::initialize<std::tuple<at::Tensor,
at::Tensor, at::Tensor, at::Tensor, at::Tensor, at::Tensor, at::Tensor, at::Tensor, a
t::Tensor> (*&)(at::Tensor const&, at::Tensor const&, at::Tensor const&, at::Tensor con
st&, at::Tensor const&, at::Tensor const&, float, at::Tensor const&, at::Tensor const&,
at::Tensor const&, float, float, at::Tensor const&, at::Tensor const&, int, at::Tensor
const&, at::Tensor const&, int, at::Tensor const&, at::Tensor const&, at::Tensor const
&, at::Tensor const&), std::tuple<at::Tensor, at::Tensor, at::Tensor, at::Tensor, at::T
ensor, at::Tensor, at::Tensor, at::Tensor, at::Tensor>, at::Tensor const&, at::Tensor c
onst&, at::Tensor const&, at::Tensor const&, at::Tensor const&, at::Tensor const&, floa
t, at::Tensor const&, at::Tensor const&, at::Tensor const&, float, float, at::Tensor co
nst&, at::Tensor const&, int, at::Tensor const&, at::Tensor const&, int, at::Tensor con
st&, at::Tensor const&, at::Tensor const&, at::Tensor const&, pybind11::name, pybind1
1::scope, pybind11::sibling>(std::tuple<at::Tensor, at::Tensor, at::Tensor, at::Tensor,
at::Tensor, at::Tensor, at::Tensor, at::Tensor, at::Tensor> (*&)(at::Tensor const&, a
t::Tensor const&, at::Tensor const&, at::Tensor const&, at::Tensor const&, at::Tensor c
onst&, float, at::Tensor const&, at::Tensor const&, at::Tensor const&, float, float, a
t::Tensor const&, at::Tensor const&, int, at::Tensor const&, at::Tensor const&, int, a
t::Tensor const&, at::Tensor const&, at::Tensor const&, at::Tensor const&), std::tuple<
at::Tensor, at::Tensor, at::Tensor, at::Tensor, at::Tensor, at::Tensor, at::Tensor, a
t::Tensor, at::Tensor> (*)(at::Tensor const&, at::Tensor const&, at::Tensor const&, a
t::Tensor const&, at::Tensor const&, at::Tensor const&, float, at::Tensor const&, at::T
ensor const&, at::Tensor const&, float, float, at::Tensor const&, at::Tensor const&, in
t, at::Tensor const&, at::Tensor const&, int, at::Tensor const&, at::Tensor const&, a
t::Tensor const&, at::Tensor const&), pybind11::name const&, pybind11::scope const&, py
bind11::sibling const&)::{lambda(pybind11::detail::function_call&)#3}::operator()(pybin
d11::detail::function_call&) const [clone .isra.0] [0x2d170e] in _C.cpython-310-x86_64-
linux-gnu.so
```

```
=========     Host Frame: pybind11::cpp_function::dispatcher(_object*, _object*, _o
bject*) [0x2cdd9c] in _C.cpython-310-x86_64-linux-gnu.so

=========     Host Frame: cfunction_call in methodobject.c:543 [0x13bf45] in python

=========     Host Frame: PyObject_Call in call.c:317 [0x149bce] in python

=========     Host Frame: _PyEval_EvalFrameDefault in ceval.c:4277 [0x131a05] in py
thon

=========     Host Frame: _PyFunction_Vectorcall in call.c:342 [0x13c41b] in python

=========     Host Frame: _PyEval_EvalFrameDefault in ceval.c:4277 [0x12ed38] in py
thon

=========     Host Frame: method_vectorcall in classobject.c:83 [0x149257] in pytho
n

=========     Host Frame: torch::autograd::PyNode::apply(std::vector<at::Tensor, st
d::allocator<at::Tensor> >&&) [0x7a8a0d] in libtorch_python.so

=========     Host Frame: torch::autograd::Node::operator()(std::vector<at::Tensor,
std::allocator<at::Tensor> >&&) [0x54fb08e] in libtorch_cpu.so

=========     Host Frame: torch::autograd::Engine::evaluate_function(std::shared_pt
r<torch::autograd::GraphTask>&, torch::autograd::Node*, torch::autograd::InputBuffer&,
std::shared_ptr<torch::autograd::ReadyQueue> const&) [0x54f565d] in libtorch_cpu.so

=========     Host Frame: torch::autograd::Engine::thread_main(std::shared_ptr<torc
h::autograd::GraphTask> const&) [0x54f6458] in libtorch_cpu.so

=========     Host Frame: torch::autograd::Engine::thread_init(int, std::shared_ptr
<torch::autograd::ReadyQueue> const&, bool) [0x54ec8a9] in libtorch_cpu.so

=========     Host Frame: torch::autograd::python::PythonEngine::thread_init(int, s
td::shared_ptr<torch::autograd::ReadyQueue> const&, bool) [0x7a11e1] in libtorch_pytho
n.so

=========     Host Frame:  [0xdc252] in libstdc++.so.6

=========     Host Frame: start_thread in pthread_create.c:442 [0x94ac2] in libc.s
o.6

=========     Host Frame:  [0x1268bf] in libc.so.6

=========     Host Frame: backward in __init__.py:132

=========     Host Frame: apply in function.py:315

=========
```

▼ 于是我们具体检查 `diff-gaussian-rasterization-w-depth_sem_gauss/cuda_rasterizer/backward.cu` 的代码, 分析发现在 line464 附近存在一个没有被边界检查的显存写入!!

```
// Backward version of the rendering procedure.

// uint32_t L 表示label的种类(使用label数量作为通道数)
```

```
template <uint32_t C, uint32_t L>

__global__ void __launch_bounds__(BLOCK_X * BLOCK_Y)

renderCUDA(

    const uint2* __restrict__ ranges,

    const uint32_t* __restrict__ point_list,

    int W, int H,

    const float* __restrict__ bg_color,

    const float2* __restrict__ points_xy_image,

    const float4* __restrict__ conic_opacity,

    const float* __restrict__ colors,

    const float* __restrict__ final_Ts,

    const uint32_t* __restrict__ n_contrib,

    const float* __restrict__ dL_dpixels,

    float3* __restrict__ dL_dmean2D,

    float4* __restrict__ dL_dconic2D,

    float* __restrict__ dL_dopacity,

    float* __restrict__ dL_dcolors,

    // semantic

    const float* __restrict__ semantics,

    const float* __restrict__ dL_dpixels_sems,

    float* __restrict__ dL_dsemantics

    )

{

    // We rasterize again. Compute necessary block info.

    auto block = cg::this_thread_block();

    const uint32_t horizontal_blocks = (W + BLOCK_X - 1) / BLOCK_X;

    const uint2 pix_min = { block.group_index().x * BLOCK_X, block.group_index().y * BL
OCK_Y };

    const uint2 pix_max = { min(pix_min.x + BLOCK_X, W), min(pix_min.y + BLOCK_Y , H)
};
```

```
    const uint2 pix = { pix_min.x + block.thread_index().x, pix_min.y + block.thread_in
dex().y };

    const uint32_t pix_id = W * pix.y + pix.x;

    const float2 pixf = { (float)pix.x, (float)pix.y };



    const bool inside = pix.x < W&& pix.y < H;

    const uint2 range = ranges[block.group_index().y * horizontal_blocks + block.group_
index().x];



    const int rounds = ((range.y - range.x + BLOCK_SIZE - 1) / BLOCK_SIZE);



    bool done = !inside;

    int toDo = range.y - range.x;



    __shared__ int collected_id[BLOCK_SIZE];

    __shared__ float2 collected_xy[BLOCK_SIZE];

    __shared__ float4 collected_conic_opacity[BLOCK_SIZE];

    __shared__ float collected_colors[C * BLOCK_SIZE];



    // semantic

    __shared__ float collected_semantics[L * BLOCK_SIZE];



    // In the forward, we stored the final value for T, the

    // product of all (1 - alpha) factors.

    const float T_final = inside ? final_Ts[pix_id] : 0;

    float T = T_final;



    // We start from the back. The ID of the last contributing

    // Gaussian is known from each pixel from the forward.

    uint32_t contributor = toDo;
```

```
const int last_contributor = inside ? n_contrib[pix_id] : 0;


float accum_rec[C] = { 0 };
float dL_dpixel[C];
// semantic
float accum_rec_sem[L] = { 0 };
float dL_dpixel_sem[L];


if (inside)
    for (int i = 0; i < C; i++)
        dL_dpixel[i] = dL_dpixels[i * H * W + pix_id];
    // semantic
    for (int i = 0; i < L; i++)
        dL_dpixel_sem[i] = dL_dpixels_sems[i * H * W + pix_id];


float last_alpha = 0;
float last_color[C] = { 0 };
// semantic
float last_semantic[L] = { 0 };


// Gradient of pixel coordinate w.r.t. normalized
// screen-space viewport corrdinates (-1 to 1)
const float ddelx_dx = 0.5 * W;
const float ddely_dy = 0.5 * H;


// Traverse all Gaussians
for (int i = 0; i < rounds; i++, toDo -= BLOCK_SIZE)
{
    // Load auxiliary data into shared memory, start in the BACK
```

```
        // and load them in revers order.

        block.sync();

        const int progress = i * BLOCK_SIZE + block.thread_rank();

        if (range.x + progress < range.y)

        {

            const int coll_id = point_list[range.y - progress - 1];

            collected_id[block.thread_rank()] = coll_id;

            collected_xy[block.thread_rank()] = points_xy_image[coll_id];

            collected_conic_opacity[block.thread_rank()] = conic_opacity[coll_id];

            for (int i = 0; i < C; i++)

                collected_colors[i * BLOCK_SIZE + block.thread_rank()] = colors[coll_id
* C + i];

            // semantic

            for (int i = 0; i < L; i++)

                collected_semantics[i * BLOCK_SIZE + block.thread_rank()] = semantics[c
oll_id * L + i];



        }

        block.sync();



        // Iterate over Gaussians

        for (int j = 0; !done && j < min(BLOCK_SIZE, toDo); j++)

        {

            // Keep track of current Gaussian ID. Skip, if this one

            // is behind the last contributor for this pixel.

            contributor--;

            if (contributor >= last_contributor)

                continue;



            // Compute blending values, as before.
```

```
            const float2 xy = collected_xy[j];

            const float2 d = { xy.x - pixf.x, xy.y - pixf.y };

            const float4 con_o = collected_conic_opacity[j];

            const float power = -0.5f * (con_o.x * d.x * d.x + con_o.z * d.y * d.y) - c
on_o.y * d.x * d.y;

            if (power > 0.0f)

                continue;



            const float G = exp(power);

            const float alpha = min(0.99f, con_o.w * G);

            if (alpha < 1.0f / 255.0f)

                continue;



            T = T / (1.f - alpha);

            const float dchannel_dcolor = alpha * T;



            // Propagate gradients to per-Gaussian colors and keep

            // gradients w.r.t. alpha (blending factor for a Gaussian/pixel

            // pair).

            float dL_dalpha = 0.0f;

            const int global_id = collected_id[j];

            for (int ch = 0; ch < C; ch++)

            {

                const float c = collected_colors[ch * BLOCK_SIZE + j];

                // Update last color (to be used in the next iteration)

                accum_rec[ch] = last_alpha * last_color[ch] + (1.f - last_alpha) * accu
m_rec[ch];

                last_color[ch] = c;



                const float dL_dchannel = dL_dpixel[ch];
```

```
                dL_dalpha += (c - accum_rec[ch]) * dL_dchannel;

                // Update the gradients w.r.t. color of the Gaussian.

                // Atomic, since this pixel is just one of potentially

                // many that were affected by this Gaussian.

                atomicAdd(&(dL_dcolors[global_id * C + ch]), dchannel_dcolor * dL_dchan
nel);

            }

            // semantic

            for (int ch = 0; ch < L; ch++)

            {

                const float l = collected_semantics[ch * BLOCK_SIZE + j];

                accum_rec_sem[ch] = last_alpha * last_semantic[ch] + (1.f - last_alpha)
* accum_rec_sem[ch];

                last_semantic[ch] = l;



                const float dL_dchannel_sem = dL_dpixel_sem[ch];

                dL_dalpha += (l - accum_rec_sem[ch]) * dL_dchannel_sem;



                atomicAdd(&(dL_dsemantics[global_id * L + ch]), dchannel_dcolor * dL_dc
hannel_sem);

            }



            dL_dalpha *= T;

            // Update last alpha (to be used in the next iteration)

            last_alpha = alpha;



            // Account for fact that alpha also influences how much of

            // the background color is added if nothing left to blend

            float bg_dot_dpixel = 0;

            for (int i = 0; i < C; i++)
```

```
                bg_dot_dpixel += bg_color[i] * dL_dpixel[i];

            dL_dalpha += (-T_final / (1.f - alpha)) * bg_dot_dpixel;




            // Helpful reusable temporary variables

            const float dL_dG = con_o.w * dL_dalpha;

            const float gdx = G * d.x;

            const float gdy = G * d.y;

            const float dG_ddelx = -gdx * con_o.x - gdy * con_o.y;

            const float dG_ddely = -gdy * con_o.z - gdx * con_o.y;


            // Update gradients w.r.t. 2D mean position of the Gaussian

            atomicAdd(&dL_dmean2D[global_id].x, dL_dG * dG_ddelx * ddelx_dx);

            atomicAdd(&dL_dmean2D[global_id].y, dL_dG * dG_ddely * ddely_dy);


            // Update gradients w.r.t. 2D covariance (2x2 matrix, symmetric)

            atomicAdd(&dL_dconic2D[global_id].x, -0.5f * gdx * d.x * dL_dG);

            atomicAdd(&dL_dconic2D[global_id].y, -0.5f * gdx * d.y * dL_dG);

            atomicAdd(&dL_dconic2D[global_id].w, -0.5f * gdy * d.y * dL_dG);


            // Update gradients w.r.t. opacity of the Gaussian

            atomicAdd(&(dL_dopacity[global_id]), G * dL_dalpha);

        }

    }

}
```

▼ 具体的报错位置就是

```
 if (inside)
     for (int i = 0; i < C; i++)
         dL_dpixel[i] = dL_dpixels[i * H * W + pix_id];
     // semantic
```

```
        for (int i = 0; i < L; i++)
            dL_dpixel_sem[i] = dL_dpixels_sems[i * H * W + pix_id];
```

- 具体来说作者在原有的 `diff-gaussian-rasterization` 库的基础上添加语义维度时, 忘记了 C++ 的循环体必须要 {} 包围, 导致下一个关于语义的写入没有根据 inside 条件进行边界检查, 导致了严重的非法写入.

分析

- 为什么作者自己运行的时候没有发现这个问题
  - pytorch 版本不同步, 可能导致内存分配策略不一样, 非法内存访问通常仅在写了一个未分配的页等情况才会实际报错, 可能作者使用的低版本 pytorch 倾向于分配利用率低的更大的页, 因此在整个运行中运气好避免了报错?
  - 作者使用的分辨率恰好都是 16 的倍数? 但这应该也是不正确的
  - NVCC 版本差异? 可能不同版本的编译器采用了不同的策略?

输入：

1. 根据库要求安装环境
2. 根据readme运行GS-SLAM代码
3. 选择PSNR和ATE RMSE作为评价指标
4. 选择非GT语义进行测试
5. 选择新场景进行测试
6. 尝试输出SIBR结构数据
7. Goat-core数据读取，重新匹配，以及框架适配

问题：

1. requirements.txt中的库不全；
2. 40/50系架构在适配环境时不再支持低版本torch和cuda，运行报错；
3. 使用的xformers加速库不支持50系新显卡；
4. 安装第三方依赖库时编译出错；
5. 运行时报错cuda out of memory，显存不足
6. 部分场景eval时进程被系统杀死，内存不足
7. 运行得到的结果中包含各关键帧的depth和渲染图，数量大，未统计
8. 原始replica数据集中不应该存在semantic真值，应该使用dinov2进行分割得到非GT的semantic结果
9. 原始采用GT语义进行渲染，无法拓展到新场景下
10. 分辨率与相机内参不匹配，运行渲染结果大面积错误
11. SemGauss输出缺少，和SIBR的要求不匹配
12. mIoU数值有问题，过分偏低，但实际对应效果很好
13. Goat-core的depth数据使用npy存储，需要重新适配；Goat-core使用四元数表示旋转，需要转化为位姿矩阵中的旋转矩阵；Goat-core位姿数据的坐标轴使用RUB，replica使用RDF，需要进行坐标转换
14. Goat-core数据在不使用真值位姿的情况下会发生很大的坐标漂移
15. Goat-core数据得到的rendered_semantic效果近似于随机化，语义效果在此数据集上对定位和建图基本无帮助

解决：

1. 重新统计得到所有依赖库，选择torch2.9.1+cu128版本，cuda12.8版本及编译工具，得到新的requirements.txt；
2. 在requirements.txt安装时手动去除第三方库及xformers库，
3. 使用命令 `pip3 install -U xformers --index-url https://download.pytorch.org/whl/cu129` 安装cu129版本即可；
4. 编译第三方库时添加—no-build-isolation选项即可，防止独立环境中没有torch；在编译dinov2时选择—no-deps防止改动torch版本；
5. 改动等比例降低图片分辨率由1200*680至960*544，同时更改configs中的相机参数设置以保证位置的准确性；优化sem_guass.py运行逻辑，去除不必要的计算图以尽可能增大分辨率;
6. 改动eval逻辑，取消最占内存的全MESH生成，只输出渲染图和指标
7. 编写脚本gifs和eval以得到评价指标随关键帧变化曲线、平均值及渲染结果的gif动图
8. semantic真值来源于人工标定，可以提高在边缘和极端变化情况下semantic的对应以达到更好的效果看，但效果不明显;定位真值并不关键，在semantic帮助下定位较为准确
9. 使用dinov2得到非GT的语义分类和特征信息作为优化参考值，并需要相应调节运行参数如关键帧密度、参考关键帧数量、tracking迭代次数、mapping迭代次数等以得到较好效果；实际运行过程中，渲染时会和rgb一样得到semantic的渲染图，此可以直接和GT或dinov2得到的特征成为feature loss；在经过dinov2的分类头后也可以与GT或dinov2得到的分类计算CrossEntropyLoss成为semantic loss
10. 改动分辨率使与内参维持等比例一致
11. 编写脚本从点云param.npz，相机运动轨迹traj.txt，相机内参replica.yaml生成cameras.json和point_cloud.ply以适配SIBR的要求
12. 在eval中，作者使用类别重排的gt_semantic作为mIoU的参考对象，但在算法中不使用gt_semantic的情况下，应该使用dinov2得到的gt_rgb图的分割特征作为mIoU的参考对象，修改后mIoU正常显示为80%左右
13. 参考在datasets中编写goat-core.py以读取goat-core数据并转化为正确的能够被semgauss使用的数据集，同时需要写configs中goat-core.yaml的相机内参文件和goat-core.py的运行配置文件
14. 使用真实位姿进行查看（可行，在比较时也是与SplaTAM使用真实位姿的结果进行比较）
15. 需要专门训练分割头（todo）

运行性能

- RTX 4090 (48G) 上, 进行 **全分辨率** 测试
  - 总运行时间大概 2h 11min
  - 显存占用 ~22G, 并不是主要瓶颈
  - 内存消耗峰值约 68.2G, 远超常规消费级内存的占用
- 在RTX5070上
  - 约3-4秒完成一普通帧的渲染添加、tracking连续位姿估计，在关键帧处约15-20秒完成渲染添加，tracking和更新优化地图；共2000帧，约3小时
  - 在完成后保存参数、执行评估eval，约8分钟；执行gifs生成GT和渲染后的动图约2分钟

汇总运行结果：

在Replica数据集上

在无GT语义的情况下

场景(无GT语义)	平均	room0	room1	room2	office0	office1	office2

PSNR	35.26	31.35	34.58	35.75	39.46	40.02	33.25
ATE RMSE /cm	0.36	0.37	0.55	0.26	0.31	0.23	0.26
mIoU/%	80.1	83.5	76.3	81.0	84.2	87.7	76.9
网格重建 precision/%	89.94	90.61	93.08	92.25	91.88	94.11	85.48
recall/%	78.66	82.83	83.97	80.07	76.40	72.34	76.88
f-score	83.85	0.8655	0.8829	0.8573	0.8343	0.8180	0.8095
平均建图时间 Mapping Time/ms	122	114	120	112	114	130	128

阈值 1cm 以内算匹配，precision=重建出来的点里落在 GT 1cm内的比例 ，recall=GT mesh中被重建覆盖的比例。

在有GT语义辅助的情况下

- PSNR的平均值为31.81
- ATE RMSE的平均值为0.28cm
- mIoU的平均值为41.7%
- 平均建图时间105ms
- 网格重建评估结果为
  - precision = 91.39%，recall = 83.37%，f-score = 0.8720

在Goat-core数据集上

场景(无GT语义)（前50帧）	平均	4ok	5cd	nfv	tee	5cd（全量）	5cd（无(姿）
PSNR	29.73	29.18	29.24	31.57	28.94	20.29	18.65
ATE RMSE /cm	-	-	-	-	-	-	855.87

不足：

1. 运行过程中语义分割边界不清，语义分割散乱受光线角度影响大，有时语义分割存在错误
2. 渲染时物体边界及细节点处存在黑边、黑点、黑块
3. 相机位姿变化过快或分辨率较低时渲染精度大幅下降
4. 分辨率与相机内参不匹配时运行渲染结果大面积错误，即相机内参对结果影响很大
5. 未知视角渲染图像效果可见未渲染的黑斑、黑块
6. 在goat-core数据集上语义分割的效果约等于没有，语义的功能消失了，和不带语义的SplaTAM baseline的效果相近
7. 在goat-core数据集上长时间范围内建图效果相比短时间显著下降

原因分析：

1. DINOV2 在场景的泛化有限，且没有 GT 语义约束，语义特征噪声直接进入优化，边界处最容易被混淆，光线不足导致细节缺失时语义分割困难。
2. 位姿估计偏差 + 语义/几何噪声会在高斯更新时累积；同时 pruning 或新增高斯阈值不合适，会让局部区域密度不足或异常。
3. 跟踪依赖连续帧的小位姿变化和足够的像素细节，帧间视差过大或下采样会让优化难以收敛，误差被后续映射放大。

4. 相机内参决定像素到射线的几何映射，缩放比例不一致会导致投影系统性偏移，渲染会整体错位甚至崩坏。
5. 相机的运行及图片并不能覆盖所有区域，部分区域缺少高斯球渲染或优化
6. dinov2并不包含分割头，需要专门训练分割头把dinov2模型输出的patch转化为分割好的图片结果，语义分割的模型需要在专门数据集上训练才可以，不能直接把replica的模型用于goat-core数据集的semantic分割
7. 连续帧间重叠程度不足初次优化不够，在之后过程中再次见到物体优化时互相影响使效果下降